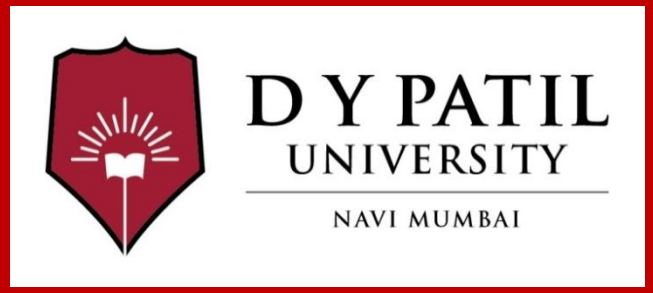
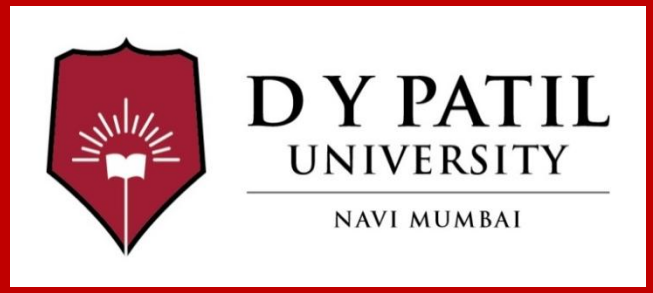


Subject Name : Web Development with Java

Unit No: 3

Unit Name :Java Servlet Pages

Faculty Name :Pooja Mehta



Unit No.:3

Faculty Name : Pooja Mehta

Contents

JSP

- Introduction to JSP and its role in web applications
- Comparison with servlet
- JSP architecture
- JSP programming structure
- JSP scripting elements
- JSP directives
- JSP actions
- Implicit objects in JSP
- Developing JSP applications with HTML
- Request and response handling in JSP
- Introduction to JavaBeans and use of beans with JSP
- Integration of JSP with servlet (MVC Application)

Build and Dependency Management using Maven

- Introduction to Maven and its role in web applications
- Maven project structure
- pom.xml file and dependency management

Reference Book:

Core Servlets and Java Server Pages: Vol I: Core Technologies 2/e , Marty Hall and Larry Brown, Pearson

Index

What is Java Server Pages (JSP)?

- Java Server Pages (JSP) is a technology for developing web pages that support dynamic content.
- It helps to insert java code in HTML pages by making use of special JSP tags.

Example

<% ...JSP Tag... %>

- JSP is a server-side program that is similar in design and functionality to java servlet.
- A JSP page consists of HTML tags and JSP tags.
- JSP pages are saved with .jsp extension

JSP	Servlets
1. JSP is a webpage scripting language, generally used to create the dynamic web content	1. Servlets are Java programs that are already compiled and which also create dynamic web content
2. JSP is an interface on top of Servlets and it is "translated" into java code	2. servlet is server-side program
3. JSP acts as a viewer	3. Servlets acts as a controller
4. JSP runs slower than Servlet, since jsp compiles into servlet code.	4. Servlet runs faster than JSP
5. The program of jsp compiled into a java servlet before execution	5. Servlet Executes inside a web server like tomcat.
6. JSP can build custom tags which can directly call java beans.	6. In servlet no such facility available.
7. It has implicit objects	7. No implicit objects
8. It is easier to write code in JSP	8. It is not easy to write code than JSP.

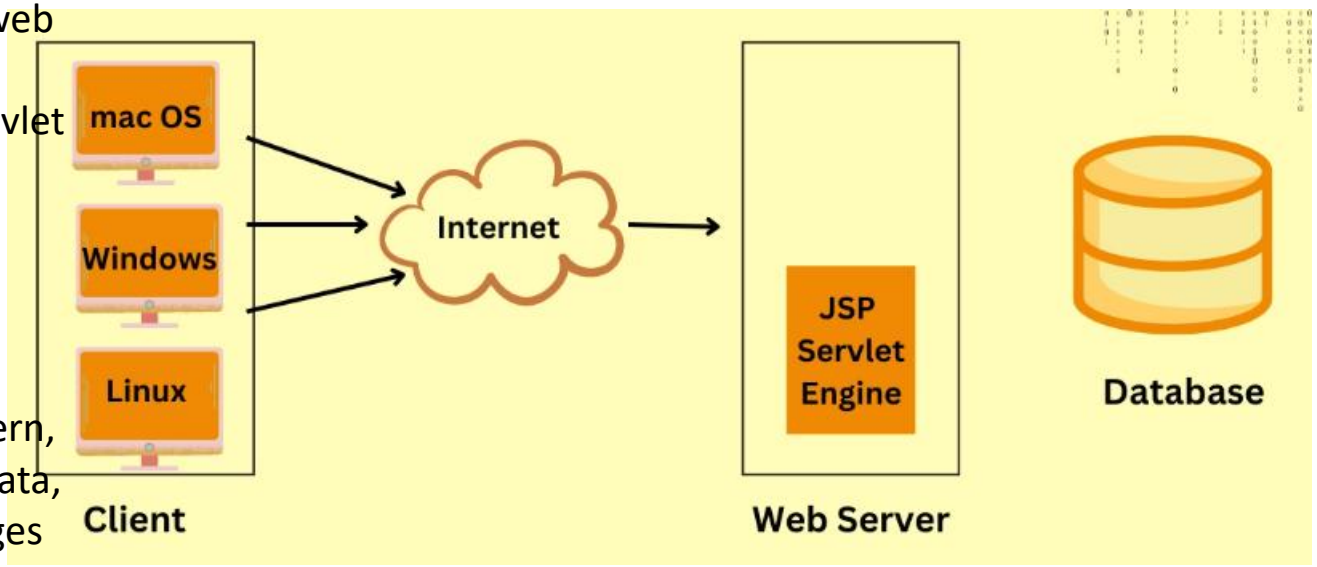
Advantages of JSP over Servlets

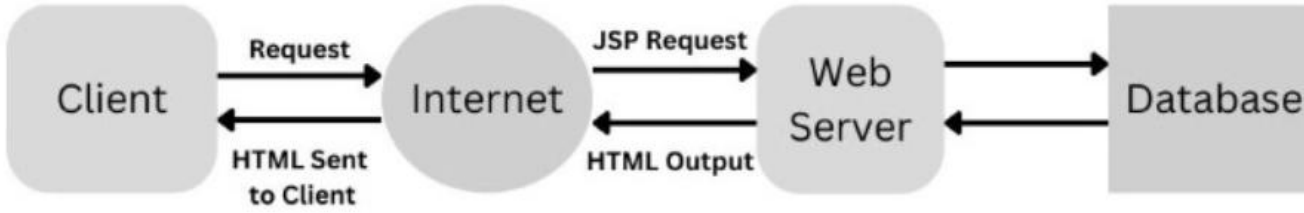
- In a JSP page visual content and logic are separated, which is not possible in a servlet.i.e. JSP separates business logic from the presentation logic.

- Servlets use println statements for printing an HTML document which is usually very difficult to use. JSP has no such tedious task to maintain.

JSP Architecture

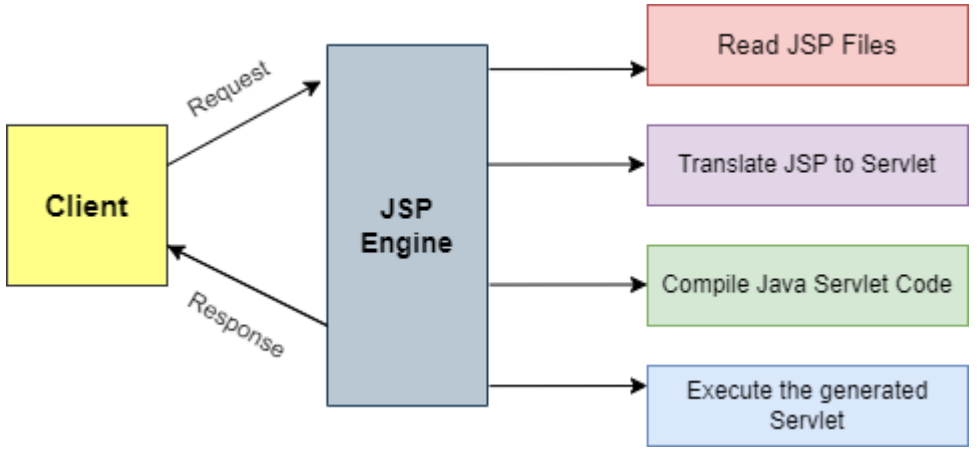
- JSP (JavaServer Pages) architecture is a framework for building web applications in Java.
- It has three main parts: a web container, a JSP engine, and a servlet container.
- The **JSP engine** processes requests from the user and generates dynamic content using Java code and JSP tags.
- The servlet container manages the lifecycle of servlets and JSP pages.
- JSP architecture follows the Model-View-Controller (MVC) pattern, where the Model contains the application's business logic and data, the View displays the data to the user, and the Controller manages the interaction between the Model and the View.
- JSP architecture provides a scalable and efficient way to build complex web applications in Java.
- JSP architecture is a **three-tier architecture** consisting of a web server, client, and database.





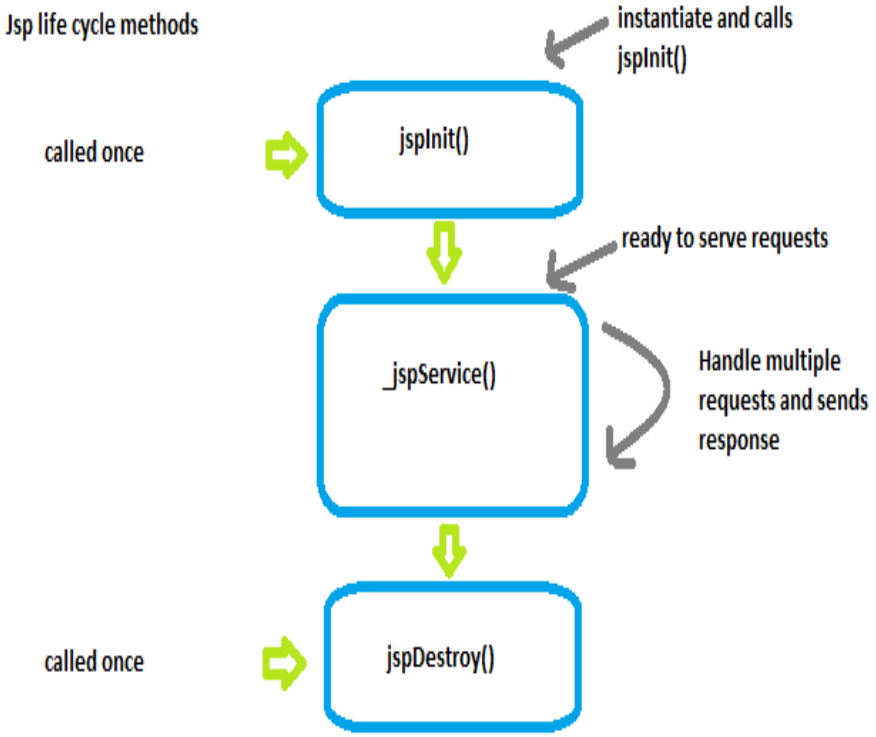
D Y PATIL UNIVERSITY
NAVI MUMBAI

- **Web Server**
- It uses a JSP engine like a container that processes JSP.
- E.g., Apache Tomcat. This engine intercepts the request for JSP and provides a runtime environment for the understanding of JSP processing files
- . It reads, parses, builds Java Servlet, Compiles, Executes Java code,
- and returns the HTML page to the client.
- **Client**
- It is the web browser/application on the user's side.
- **Database**
- It is used to store the user's data. The web server has access to the database.



Life Cycle of JSP

- A JSP life cycle can be defined as the entire process from its creation till the destruction.
- It is similar to a servlet life cycle with an additional step which is required to compile a JSP into servlet.
- A JSP page is converted into Servlet in order to service requests.
- The translation of a JSP page to a Servlet is called Lifecycle of JSP.



D Y PATIL UNIVERSITY
NAVI MUMBAI

jspInit()

- This is declared on the JSP page. This method is called only once in the complete lifecycle of JSP. This is used to initialize configuration params in a deployment descriptor. This method can be overridden using a JSP declaration scripting element.

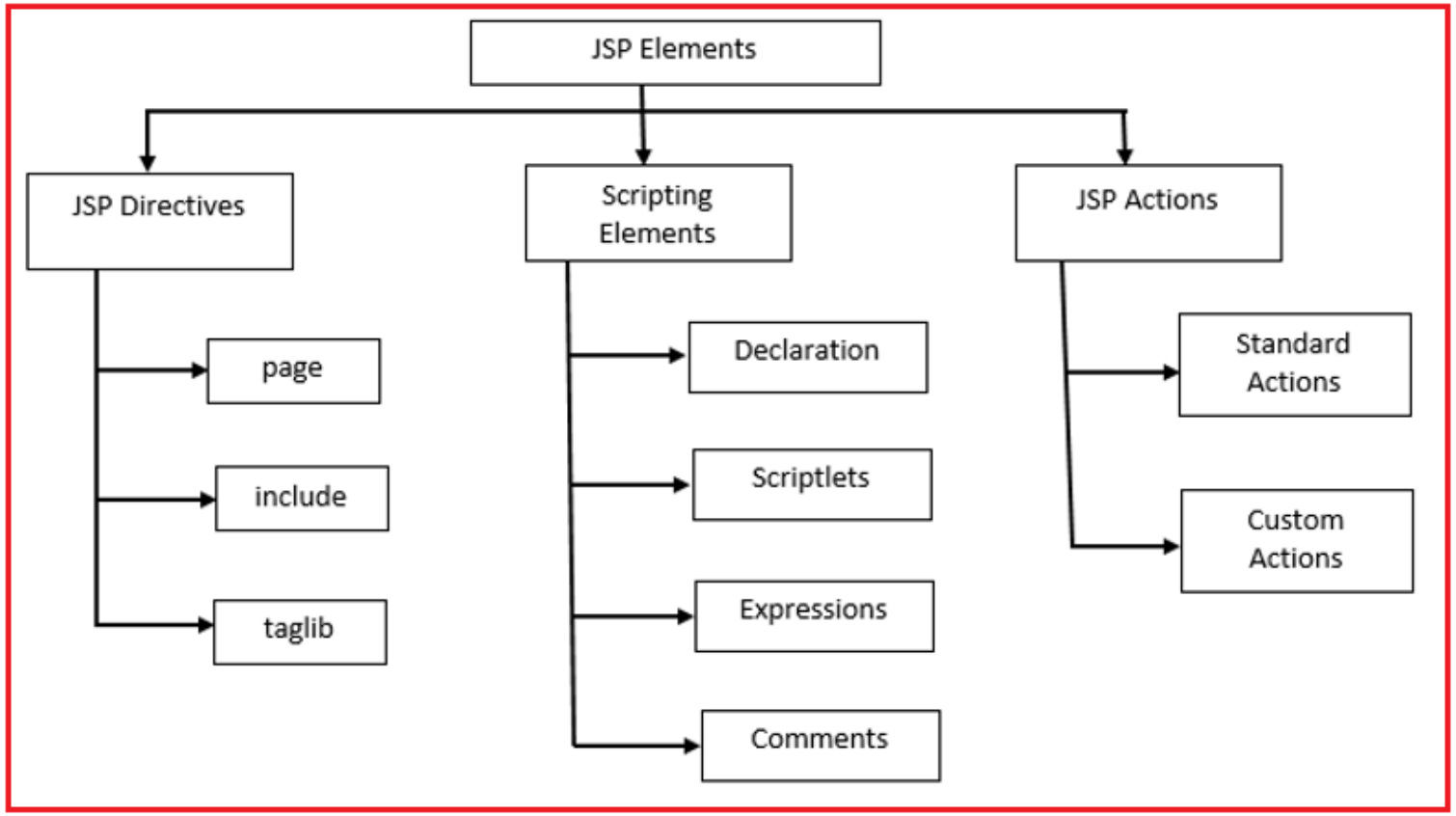
_jspService()

- It is invoked by the JSP container for each client request. This method passes the request and response objects. It cannot be overridden, and hence, it starts with an underscore. It is defined and declared in the HttpJspPage interface.

jspDestroy()

- This is used for shutting down the application/container. This method is called only once in the complete JSP lifecycle when JSP is unloaded from the memory. This method should be overridden only to release resources created in the JSP init method.

JSP Elements



JSP Scripting Elements

- The scripting elements provides the ability to insert java code inside the jsp. There are three types of traditional scripting elements:
  - scriptlet tag
  - expression tag
  - declaration tag

scriptlet tag

- A scriptlet tag is used to execute java source code in JSP.
- A scriptlet can contain Any number of JAVA language statements, Variable, Method declarations
- Expressions that are valid in the page scripting language
- Syntax

<%

// java source code

%>

Example

<% out.print("welcome to jsp"); %>

<% int a=10; %>

- Everything written inside the scriptlet tag is compiled as java code.
- JSP code is translated to Servlet code, in which _jspService() method is executed which has HttpServletRequest and HttpServletResponse as argument.
- JSP page can have any number of scriptlets, and each scriptlets are appended in _jspService ().

First jsp program: First.jsp using scriptlet

```
<html>
<body>
<%
out.println("Hello World! My First JSP Page");
%>
</body>
</html>
```

expression

- The code placed within JSP expression tag is written to the output stream of the response.So you need not write out.print() to write data.
- It is mainly used to print the values of variable or method.
- Syntax

<%=statement %>

<%= (2*5) %> --------→out.print((2*5));

- “Do not end the statement with semicolon in case of expression tag.“

declaration

- The JSP declaration tag is used to declare variables and methods
- The declaration of jsp declaration tag is placed outside the _jspService() method.

- Syntax
- <%! variable or method declaration %>

Example

<%! int a = 10; %>

<%! int a, b, c; %>

<%! Circle a = new Circle(2.0); %>

comments

- The comments can be used for documentation.
- This JSP comment tag tells the JSP container to ignore the comment part from compilation.

- Syntax

- <%-- comments --%>

Example

```
<html>

<body>

<%-- comment:JSP Scipting elements --%>

<%! int i=0; %>

<% i++; %>

Welcome to world of JSP!

<%= "This page has been accessed " + i + " times" %>

</body>

</html>
```

JSP Directives

JSP Directives control the processing of an entire JSP page. It gives directions to the server regarding processing of a page. There are three types of directives available in JSP: page, include and taglib.

```
<%@ directive name [attribute name="value" attribute name="value" ........]%>
```

–

Page Directive

There are several attributes, which are used along with Page Directives and these are

- import
- session
- isErrorPage
- errorPage
- ContentType
- isThreadSafe
- extends
- info
- language
- autoflush
- Buffer

Example :import, contentType,language

```
<%@ page language="java" contentType="text/html" import="java.util.Date" %>
<html>
<body>
<%
// Printing current date using imported Date class
out.println("Current Date: " + new Date());
%>
</body>
</html>
```

Include Directive

- The Include Directive helps to include content from another file (whether static or dynamic) within the current JSP page. This happens at translation time, meaning included content becomes part of the JSP before it gets compiled.

- Syntax: <%@ include file=”relativeURL” %>

```
<%@ include file="header.jsp" %>
<html>
<body>
<h2>This is the main content of the page.</h2>
<%@ include file="footer.jsp" %>
</body>
</html>
```

isErrorPage
errorPage

```
<%@ page language="java"
contentType="text/html; charset=ISO-8859-1"
   pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
<form action="process.jsp">
   Number 1: <input type="text" name="n1"
/><br/><br/>
   Number 2: <input type="text" name="n2"
/><br/><br/>
   <input type="submit" value="Divide" />
</form>
</body>
</html>
```

```
// process.jsp
<%@ page errorPage="error.jsp" %>
<%
    String num1 =
request.getParameter("n1");
    String num2 =
request.getParameter("n2");

    int a =
Integer.parseInt(num1);
    int b =
Integer.parseInt(num2);
    int c = a / b; // May throw
ArithmeticException if b is 0

    out.print("The result of the
division is: " + c);
%>
```

```
//Error.jsp
<%@ page isErrorPage="true"
%>
<h3>Oops! Something went
wrong.</h3>
<p>Error Details: <%=
exception.getMessage() %></p>
```

- The **isErrorPage**=”**true**” allows this page to access the **exception** object, which contains information about the error.
- This page shows a friendly message and the details of the exception.

isThreadSafe and buffer

- isThreadSafe as true, it means that the JSP page supports multithreading. On the other hand if it is set to false then JSP engine won't allow multithreading which means only single thread will execute the page code.
- Default value for isThreadSafe attribute: true.

Syntax of isThreadSafe attribute:

- <%@ page isThreadSafe="value"%>

buffer:
This attribute is used to specify the buffer size. If you specify this to none during coding then the output would directly written to Response object by JSPWriter. And, if you specify a buffer size then the output first written to buffer then it will be available for response object.

Syntax of buffer attribute:

<%@ page buffer="value"%>
value is size in kb or none.

```
<html>
<body>

<%@ page buffer="30kb" %>
Current Date: <%= new java.util.Date() %>

</body>
</html>
```

extends:

Like java, here also this attribute is used to extend(inherit) the class.

Syntax of extends attribute:

```
<%@ page extends="value"%>
```

Value is package_name.class_name.

Example of extends:

The following code will inherit the SampleClass from package: `mypackage`

```
<%@ page extends="mypackage.SampleClass"%>
```

info:

It provides a description to a JSP page. The string specified in info will return when we will call `getServletInfo()` method.

Syntax of info:

```
<%@ page info="value"%>
```

here value is Message or Description

Example of info attribute:

```
<%@ page info="This code provide information"%>
```

JSP Actions

- XML format is used to define JSP Actions

jsp:useBean

jsp:include

jsp:setProperty

jsp:getProperty

jsp:forward

jsp:plugin

jsp:attribute

jsp:body

jsp:text

jsp:param

jsp:attribute

jsp:output

Action Tag	Description
jsp:forward	Used for forwarding the request and response to other resource
jsp:include	Used for including response from another resource
jsp:body	Used for defining dynamically defined body of XML element
jsp:useBean	Used for creating or locating java beans
jsp:setProperty	Used for setting property value in java bean
jsp:getProperty	Used for getting property value in java bean
jsp:plugin	Used for embedding other components/applets
jsp:element	Used for defining XML elements dynamically.
jsp:param	Used for setting the parameter for forward or include value
jsp:text	Used for writing template text in JSP pages and documents.
jsp:fallback	Used for printing the message if plugins are working
jsp:attribute	Used for defining attributes of dynamically-defined XML element

<java:include>

- This action will include the required resources like html, servlets and JSP.
- This jsp:include action is different from jsp directive. Include directive includes resources at the time of translation, whereas include action includes resources dynamically at request time. Action directives work well for static pages, whereas later works better for dynamic pages.
  There are two attributes under include:
- **Page:** its value is the url of the required resource.
- **Flush:** it checks that the buffer of the resource is flushed before it is included.

Syntax:

<jsp:include page="page URL" flush="true/false">

Example:<jsp:include page="date.jsp" flush="true" />

</body>

</html>

date.jsp

Today's date: <%= (**new** java.util.Date()).toLocaleString()%>

<jsp:forward>

- This action as the name suggests forwards the request to another page. This other page may be static as well as a JSP page.**Syntax:**
- main.jsp
- <jsp:forward =“otherpage.jsp”>
- **Example:**

Forward.jsp

```
<a>I was the requested page but forwarded the request to other</a>
<jsp:forward page="other.jsp" />
```

Other.jsp

```
<html>
<title>JSP Forward 2</title>
</head>
<body>
<a>I am the other page where request is forwarded.</a>
</body>
</html>
```

Tag	Purpose	Syntax	Example
<jsp:useBean>	Creates or finds a bean	<jsp:useBean id="id" class="pkg.Class" scope="session" />	<jsp:useBean id="user" class="com.model.User" scope="session" />
<jsp:setProperty>	Sets bean property	<jsp:setProperty name="id" property="prop" value="val" />	<jsp:setProperty name="user" property="name" value="Pooja" />
<jsp:getProperty>	Gets bean property	<jsp:getProperty name="id" property="prop" />	Name: <jsp:getProperty name="user" property="name" />
<jsp:include>	Includes another file	<jsp:include page="file.jsp" />	<jsp:include page="header.jsp" />
<jsp:forward>	Forwards request	<jsp:forward page="file.jsp" />	<jsp:forward page="home.jsp" />
<jsp:param>	Passes parameter	<jsp:param name="n" value="v" />	<jsp:forward page="home.jsp"><jsp:param name="user"value="Pooja"/></jsp:forward>
<jsp:plugin>	Embeds applet	<jsp:plugin type="applet" code="Class.class" />	<jsp:plugin type="applet" code="MyApplet.class" />
<jsp:fallback>	Message if plugin fails	<jsp:fallback>msg</jsp:fallback>	<jsp:fallback>Not supported</jsp:fallback>
<jsp:element>	Creates dynamic element	<jsp:element name="tag">...</jsp:element>	<jsp:element name="h1"> Hello</jsp:elemet>
<jsp:attribute>	Adds attribute dynamically	<jsp:attribute name="attr">val</jsp:attribute>	<jsp:attribute name="style">color:red;</jsp:attribute>
<jsp:body>	Defines body content	<jsp:body>content</jsp:body>	<jsp:body>Welcome</jsp:body>

Action Tag	Description
jsp:forward	forward the request to a new page Usage : <jsp:forward page="Relative URL" />
jsp:useBean	instantiates a JavaBean Usage : <jsp:useBean id="beanId" />
jsp:getProperty	retrieves a property from a JavaBean instance. Usage : <jsp:useBean id="beanId" ... /> ... <jsp:getProperty name="beanId" property="someProperty" .../> Where, beanName is the name of pre-defined bean whose property we want to access.

jsp:setProperty	store data in property of any JavaBeans instance. Usage : <jsp:useBean id="beanId" ... /> ... <jsp:setProperty name="beanId" property="someProperty" value="some value"/> Where, beanName is the name of pre-defined bean whose property we want to access.
jsp:include	includes the runtime response of a JSP page into the current page.
jsp:plugin	Generates client browser-specific construct that makes an OBJECT or EMBED tag for the Java Applets

jsp:fallback	Supplies alternate text if java plugin is unavailable on the client. You can print a message using this, if the included jsp plugin is not loaded.
jsp:element	Defines XML elements dynamically
jsp:attribute	defines dynamically defined XML element's attribute
jsp:body	Used within standard or custom tags to supply the tag body.
jsp:param	Adds parameters to the request object.
jsp:text	Used to write template text in JSP pages and documents. Usage : <jsp:text>Template data</jsp:text>

- **<jsp:forward>**

Purpose: Forwards request to another resource

Syntax:

<jsp:forward page="file.jsp" />

Example:

<jsp:forward page="home.jsp" />

-

- **<jsp:param>**

Purpose: Pass parameters to include/forward

Syntax:

<jsp:param name="paramName" value="value" />

Example:

```
<jsp:forward page="home.jsp">
   <jsp:param name="user" value="Pooja" />
</jsp:forward>
```

- **<jsp:plugin>**

Purpose: Embeds Java applet/component (rarely used now)

Syntax:

<jsp:plugin type="applet" code="ClassName.class" />

Example:

<jsp:plugin type="applet" code="MyApplet.class" />

<jsp:fallback>

Purpose: Displays message if plugin fails

Syntax:

<jsp:fallback>Message</jsp:fallback>

Example:

<jsp:fallback>Applet not supported</jsp:fallback>

- **<jsp:element>**

Purpose: Dynamically creates XML/HTML element

Syntax:

<jsp:element name="tagName">content</jsp:element>

Example:

<jsp:element name="h1">Hello</jsp:element>

- **<jsp:attribute>**

Purpose: Defines attribute dynamically

Syntax:

```
<jsp:attribute name="attrName">value</jsp:attribute>
```

Example:

```
<jsp:element name="h1">
   <jsp:attribute name="style">color:red;</jsp:attribute>
   Hello
</jsp:element>
```

- **<jsp:body>**

Purpose: Specifies body content for tags

Syntax:

```
<jsp:body>content</jsp:body>
```

Example:

```
<jsp:element name="p">
   <jsp:body>Welcome</jsp:body>
</jsp:element>
```

Access JavaBeans in JSP Application

- **<jsp:useBean>**

The jsp:useBean tag is utilized to start up an object of JavaBean or it can re-utilize the existing java bean object. The primary motivation behind jsp:useBean tag is to interface with bean objects from a specific JSP page

<jsp:useBean id=”—” class”—” type”—” scope=”—”/>

Attributes:

- **Id**: It will take a variable to manage generated Bean object reference.
- **Class**: This attribute will take the fully qualified name of the Bean class.
- **Type**: It will take the fully qualified name of the Bean class to define the type of variable in order to manage the Bean object reference.
- **Scope**: It will take either of the JSP scopes to the Bean object.

<jsp:getProperty>

The jsp:getProperty tag is utilized to get indicated property from the JavaBean object. The principle reason for <jsp:getProperty> tag is to execute a getter strategy to get an incentive from the Bean object.

Syntax: <jsp:getProperty name=”—” property=”—”/>

Attributes:

Name: It will take a variable that is the same as the id attribute value in <jsp:useBean> tag.

Property: It will take a particular property to execute the respective getter method.

<jsp:setProperty>

The jsp:setProperty tag is utilized to set a property in the JavaBean object. The principle motivation behind jsp:setProperty tag is to execute a specific setter technique to set an incentive to a specific Bean property.

Syntax: <jsp:setProperty name=”—” property=”—” value=”—”/>

Attributes:

Name: It will take a variable that is the same as the id attribute value in <jsp:useBean> tag.

Property: It will take a property name in order to access the respective setter method.

Value: It will take a value to pass as a parameter to the respective setter method.

Jsp Implicit Object

Implicit Object	Description
request	The HttpServletRequest object associated with the request.
response	The HttpServletRequest object associated with the response that is sent back to the browser.
out	The JspWriter object associated with the output stream of the response.
session	The HttpSession object associated with the session for the given user of request.
application	The ServletContext object for the web application.
config	The ServletConfig object associated with the servlet for current JSP page.
pageContext	The PageContext object that encapsulates the enviroment of a single request for this current JSP page
page	The page variable is equivalent to this variable of Java programming language.
exception	The exception object represents the Throwable object that was thrown by some other JSP page.

Jsp Implicit Objects: out

- For writing any data to the buffer, JSP provides an implicit object named out.
- It is an object of JspWriter.

```
//Servlet code
PrintWriter out=response.getWriter();

response.setContentType("text/html");

out.print("Welcome");
```

```
//Jsp code
<html>
<body>
<% out.print("Welcome"); %>
</body>
</html>
```

Jsp Implicit Objects: request

- Instance of javax.servlet.http.HttpServletRequest object associated with the request.
- Each time a client requests a page the JSP engine creates a new object to represent that request.
- The request object provides methods to get HTTP header information including from data, cookies, HTTP methods etc.

Request.html

```
<form action="welcome.jsp">
Login:<input type="text" name="login">
<input type="submit" value="next">
</form>
```

Welcome.jsp

```
hello,
<%
out.println(request.getParameter("login"));
%>
```

Jsp Implicit Objects: response

- The response object is an instance of a javax.servlet.http.HttpServletResponse object.
- Object the JSP programmer can add new cookies or date stamps, HTTP status codes, redirect response to another resource, send error etc.
- config is an implicit object of type javax.servlet.ServletConfig.

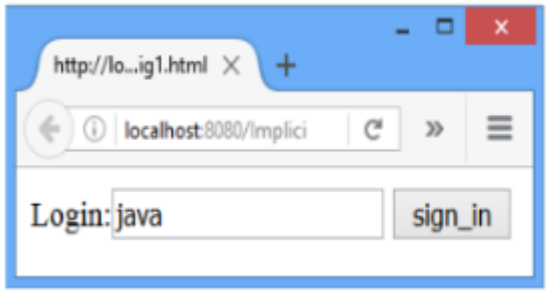
- This object can be used to get initialization parameter for a particular JSP page.

- Config.html

```
<form action="MyConfig">
Login:<input type="text" name="login">
<input type="submit" value="sign_in">
</form>
```

MyConfig.jsp

```
<%
out.print("Welcome "+request.getParameter("login"));
String c_name=config.getInitParameter("College");
out.print("<p>College name is="+c_name+"</p>");
%>
```

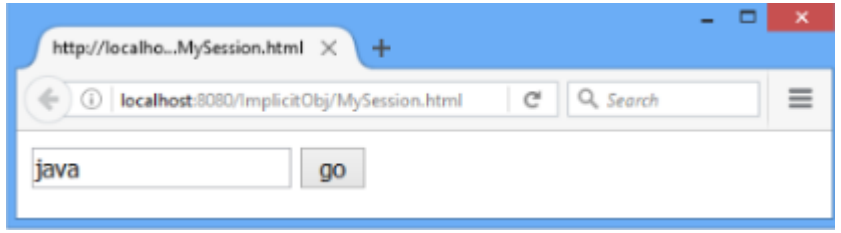


D Y PATIL UNIVERSITY
NAVI MUMBAI

Jsp Implicit Objects: session

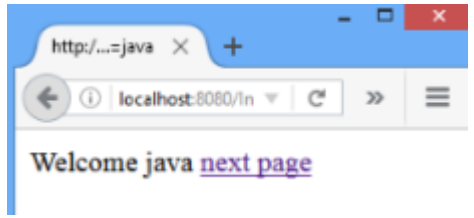
- In JSP, session is an implicit object of type javax.servlet.http.HttpSession .
- The Java developer can use this object to set, get or remove attribute or to get session information.
- MySession.html

```
<form action="MySession1.jsp">
<input type="text" name="uname">
<input type="submit" value="go"><br/>
</form>
```



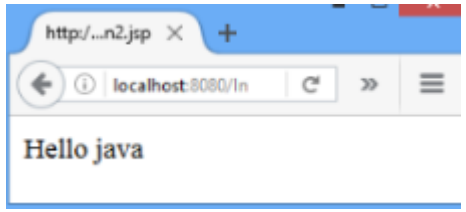
D Y PATIL UNIVERSITY
NAVI MUMBAI

```
<html>
<body>
<%
String name=request.getParameter("uname");
out.print("Welcome "+name);
session.setAttribute("user",name);
%>
<a href="MySession2.jsp">next page</a>
</body>
</html>
```



MySession2.jsp

```
<html>
<body>
<%
String name=
(String)session.getAttribute("user");
out.print("Hello "+name);
%>
</body>
</html>
```



JSP Action

- JSP actions use the construct in XML syntax to control the behavior of the servlet engine.
- can dynamically insert a file, reuse the beans components, forward user to another page, etc. through JSP Actions like include and forward.
- Unlike directives, actions are re-evaluated each time the page is accessed

Syntax:

```
<jsp:action name attribute="value" />
```

welcome.jsp

```
<html>
    <head>
        <title>Welcome Page</title>
    </head>
  <body>
        <%@ include file="header.jsp" %>
        Welcome, User
  </body>
</html>
```

header.jsp

```
<html>
  <body>
      <img src="header.jpg" alt="This is Header image" / >
  </body>
</html>
```

JSP Session and Cookies Handling

- Cookies are text files stored on the client computer and they are kept for various information tracking purpose.
- JSP transparently supports HTTP cookies using underlying servlet technology.

```
//Cookie1.jsp
<% Cookie cookie = new Cookie("c1","MyCookie1");
cookie.setMaxAge(60 * 60);
response.addCookie(cookie);
%>
<html><body>
<a href="Cookie2.jsp">Click here</a>
</body></html>
//Cookie2.jsp
<% Cookie[] c2 = request.getCookies();
for(int i = 0; i < c2.length; i++) {
out.print("<p>"+c2[i].getName()+" "+
c2[i].getValue()+"</p>");
}
%>
```

JSP Session Handling

- In JSP, session is an implicit object of type HttpSession.
- The Java developer can use this object to set, get or remove attribute or to get session information.
- In Page Directive, session attribute indicates whether or not the JSP page uses HTTP sessions.
- <%@ page session="true" %> //By default it is true

```
//Session1.jsp
<%@page session="true" %>
<% session.setAttribute("s1","DIET");%>
<html>
<body>
<a href="Session2.jsp">nextPage</a>
</body>
</html>
```

```
//Session2.jsp
<%@page session="true" %>

<% String str=(String)session.getAttribute("s1");

out.println("session="+str);%>
```

Request handling in JSP

- Request – Object of HttpServletRequest Implementation
- This is an in-built implicit object of javax.servlet.http.HttpServletRequest. It is passed as a parameter to jspService method. This instance is created each time a request is generated. Using this object we can request a parameter session, cookies, header information, server name, server port, contentType, character encoding, etc.
- The syntax to use request is:

request.methodname(“value”);

```
h2>Request Methods</h2>
 <table border="1">
<tr>
<th>Header</th><th>Header Values</th>
</tr>
<%
  HttpSession mysession = request.getSession();
  out.print("<tr><td>Session Name is </td><td>" +mysession+ "</td.></tr>");
  Locale mylocale = request.getLocale ();
  out.print("<tr><td>Locale Name is</td><td>" +mylocale + "</td></tr>");
   String method = request.getMethod();
  out.print("<tr><td>Method </td><td>" +method + "</td></tr>");
   String serverName = request.getServerName();
  out.print("<tr><td>Server Name  </td><td>" +serverName+ "</td></tr>");
  int portname = request.getServerPort();
  out.print("<tr><td>Server Port  </td><td>" +portname+ "</td></tr>");
%>
```

Request Methods

Header	Header Values
Session Name is	org.apache.catalina.session.StandardSessionFacade@33f8cf4b
Locale Name is	en_US
Method	GET
Server Name	localhost
Server Port	9090

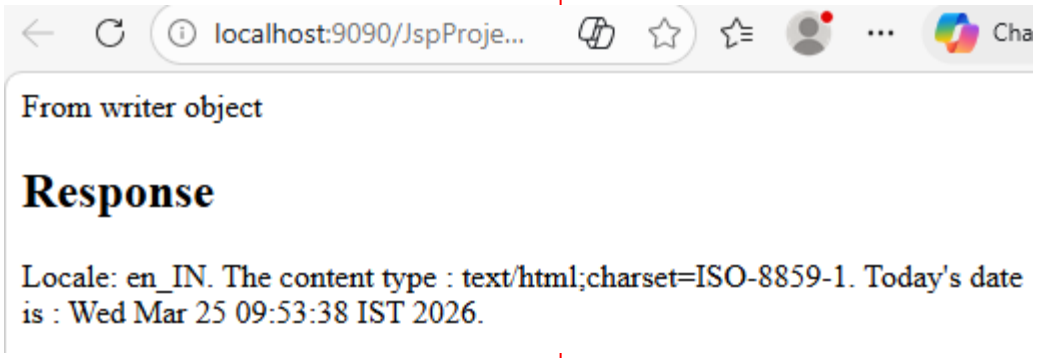
Response handling in JSP

- This is an object from HttpServletResponse. This object rather gets passed as a parameter to jspService() like request. It is the response given to the user. One can redirect, get header information, add cookies, send error messages using this object.

The Syntax for using response object is:

response.methodName();

```
<h2>Response</h2>
<%
Locale mylcl = response.getLocale();
out.println("Locale: " + mylcl + ".\n");
out.newLine();
response.flushBuffer();
out.newLine();
PrintWriter op = response.getWriter();
op.println("From writer object");
out.newLine();
String cotype = response.getContentType();
out.newLine();
out.println("The content type : " + cotype + ".\n");
out.newLine();
 response.setIntHeader("Refresh", 5);
 Date mydate = new Date();
 out.println("Today's date is : " +mydate.toString() + ".\n");
%>
```



JavaBeans in JSP

- Bean means component. Components mean reusable objects. JavaBean is a reusable component. Java Bean is a normal java class that may declare properties, setter, and getter methods in order to represent a particular user form on the server-side. JavaBean is a java class that is developed with a set of conventions.
  - The class should be public and must not be final, abstract.
  - Recommend to implement serializable.
  - Must have direct/indirect param constructor.
  - Member variables should be non-static and private.
  - Every bean property should have one setter method and one getter method with the public modifier.

Advantages of Java Beans

- It is easy to reuse the software components.
- The properties and methods of Java Bean can be exposed to another application.

Disadvantages of Java Beans

- JavaBeans are mutable objects so it cannot take advantage of immutable objects.
- JavaBeans will be in an inconsistent state because of creating the setter and getter method for each property separately.

Java Bean Properties

- **getPropertyName():** This method is used to read the property which is also called accessor.
- **setPropertyName():** This method is used to write the property which is also called mutator.

Access JavaBeans in JSP Application

- **<jsp:useBean>**

The jsp:useBean tag is utilized to start up an object of JavaBean or it can re-utilize the existing java bean object. The primary motivation behind jsp:useBean tag is to interface with bean objects from a specific JSP page

<jsp:useBean id=”—” class”—” type”—” scope=”—”/>

Attributes:

- **Id**: It will take a variable to manage generated Bean object reference.
- **Class**: This attribute will take the fully qualified name of the Bean class.
- **Type**: It will take the fully qualified name of the Bean class to define the type of variable in order to manage the Bean object reference.
- **Scope**: It will take either of the JSP scopes to the Bean object.

<jsp:getProperty>

The jsp:getProperty tag is utilized to get indicated property from the JavaBean object. The principle reason for <jsp:getProperty> tag is to execute a getter strategy to get an incentive from the Bean object.

Syntax: <jsp:getProperty name=”—” property=”—”/>

Attributes:

Name: It will take a variable that is the same as the id attribute value in <jsp:useBean> tag.

Property: It will take a particular property to execute the respective getter method.

<jsp:setProperty>

The jsp:setProperty tag is utilized to set a property in the JavaBean object. The principle motivation behind jsp:setProperty tag is to execute a specific setter technique to set an incentive to a specific Bean property.

Syntax: <jsp:setProperty name=”—” property=”—” value=”—”/>

Attributes:

Name: It will take a variable that is the same as the id attribute value in <jsp:useBean> tag.

Property: It will take a property name in order to access the respective setter method.

Value: It will take a value to pass as a parameter to the respective setter method.

```
//Test.java
package com;
public class Test {
private String message = "No message specified";
 public String getMessage() {
  return (message);
 }
 public void setMessage(String message) {
  this.message = message;
 }
}
```

```
<%@ page language="java"
contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<body>
 <h2>Using JavaBeans in JSP</h2>
 <jsp:useBean id="test" class="com.Test" />
 <jsp:setProperty name="test"
property="message" value="Hello JSP..." />
 <p>Got message....</p>
 <jsp:getProperty name="test"
property="message" />
</body>
</html>
```

Maven: Introduction

- Maven is a build automation tool used primarily for Java projects. Maven can also be used to build and manage projects written in C#, Ruby, Scala, and other languages. The Maven project is hosted by The Apache Software Foundation, where it was formerly part of the Jakarta Project.

Objective

- The primary goal of Maven is to provide a developer with a comprehensive model for projects, which is reusable, maintainable, and easy to comprehend, along with plugins or tools that interact with this declarative model.

Features of Maven:

The Maven Build Automation tool provides a lot of features to make the development easy. Below we listed them

- **Dependency Management:** Automatically downloads and manages external libraries.
- **Standard Project Structure:** Follows a fixed folder layout for source, test, and other files.
- **Build Lifecycle:** Defines standard build phases like compile, test, and deploy.
- **Plugins:** Supports plugins for compiling, testing, packaging, and more.
- **POM File:** Uses pom.xml to manage configuration and dependencies.
- **Central Repository:** Fetches dependencies from a shared online repository.
- **Build Profiles:** Supports different settings for dev, QA, and production.
- **Reporting:** Can generate Javadoc, test reports, and project documentation.
- **IDE Support:** Integrates with Eclipse, IntelliJ, NetBeans, etc.

Maven lifecycle

- Maven lifecycle and its 8 steps: Validate, Compile, Test, Package, Integration test, Verify, Install and Deploy.
- Maven lifecycle by default consists of 8 major steps or phases for compiling, testing, building and installing a given project as specified below:
- **Validate**: This step validates if the project structure is correct. For example — It checks if all the dependencies have been downloaded and are available in the local repository.
- **Compile**: It compiles the source code, converts the .java files to .class and stores the classes in target/classes folder.
- **Test**: It runs unit tests for the project.
- **Package**: This step packages the compiled code in distributable format like JAR or WAR. Integration test: It runs the integration tests for the project.
- **Verify**: This step runs checks to verify that the project is valid and meets the quality standards.
- **Install**: This step installs the packaged code to the local Maven repository.
- **Deploy**: It copies the packaged code to the remote repository for sharing it with other developers.


MAVEN
LIFE CYCLE
1 Validate
2 Compile
3 Test
4 Package
Integration test 5
Verify 6
Install 7
Deploy 8

Pom.xml

- The pom.xml (Project Object Model) file is the core configuration file for projects built with Apache Maven. It is an XML file that contains information about the project and configuration details that Maven uses to manage the project's build process, dependencies, and documentation
- The project is the root element of the pom.xml file, containing various sub-elements that define the project's details.
- <modelVersion>: Specifies the version of the Maven POM model (typically 4.0.0).
- <groupId>: A unique identifier for the project's group or organization (e.g., com.mycompany.app).
- <artifactId>: The unique name of the project or module (e.g., my-app).
- <version>: The specific version of the project (e.g., 1.0.0-SNAPSHOT).
- <dependencies>: Lists the external libraries required for the project to compile, test, or run. Each dependency is specified by its own groupId, artifactId, and version, and optionally a scope (e.g., compile, test, provided).
- <build>: Contains configuration information about the project's build process, including source directories, output directories, and plugin configurations.
- <plugins>: Specifies which plugins should be used during the build process and how they should be configured (e.g., the Maven Compiler Plugin to set the Java version).
- <properties>: Allows the definition of variables that can be reused throughout the POM, which helps manage consistent versioning and other settings (e.g., ${java.version}).
- <parent>: Allows a project to inherit configurations, dependencies, and plugins from a parent POM, which is useful for multi-module projects and standardizing settings across an organization.
- <repositories>: Defines remote repositories from which Maven can download dependencies and plugins (by default, Maven uses the Maven Central repository).

Pom.xml Structure

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com</groupId>
  <artifactId>MavenProj</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>jar</packaging>

  <name>MavenProj</name>
  <url>http://maven.apache.org</url>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>
    <!-- Source: https://mvnrepository.com/artifact/mysql/mysql-connector-java -->
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.32</version>
    <scope>compile</scope>
</dependency>
  </dependencies>
</project>
```

Thank You

D Y PATIL
UNIVERSITY
NAVI MUMBAI